# Week 8
# Advantage Actor-Critic, GAE, and TRPO

STUDY-RL — a self-paced Deep RL curriculum

October 20, 2026

# Contents

**Learning objectives** 3

**1 Motivation and intuition** 4

**2 Mathematical formulation** 5
- 2.1 $n$-step advantages . . . 5
- 2.2 Generalised Advantage Estimation (Schulman 2015) . . . 5
- 2.3 Natural Policy Gradient . . . 6
- 2.4 TRPO (Schulman et al. 2015) . . . 6

**3 Algorithms** 9

**4 Implementation walkthrough** 10
- 4.1 GAE in $\sim$10 lines . . . 10
- 4.2 A2C update . . . 10
- 4.3 Fisher–vector product (TRPO core) . . . 11

**5 Empirical results** 12

**6 Safety angle** 14

**7 Matching application** 15

**8 Exercises** 16

**9 Further reading** 17

# Learning objectives

**Study companion.** Read `teaching_guide.md` in this folder for step-by-step examples, an analogy with its limits, and self-checks. Then use `workbook.ipynb` to implement and vary the calculations.

**Key Idea**

By the end of this week you will be able to:

1. Derive $n$-step advantages and the closed-form Generalised Advantage Estimator (GAE) as a geometric mixture, and explain the bias–variance role of $\lambda$.
2. Implement synchronous A2C with GAE on LunarLander-Continuous and tune the $(\gamma, \lambda)$ pair sensibly.
3. State the natural-policy-gradient update via the Fisher information matrix, including its invariance to reparameterisation.
4. Derive TRPO end-to-end: surrogate objective, KL trust region, second-order Taylor of the constraint, Lagrangian, conjugate-gradient $F^{-1}g$ solve, and back-tracking line search, distinguishing its empirical acceptance checks from the ideal improvement guarantee.
5. Recognise when a first-order PPO-style approximation suffices and when the full TRPO machinery is worth its compute.
6. Connect KL trust regions to the alignment notion of "stay close to the reference model" that underpins RLHF/PPO and DPO.

# 1 Motivation and intuition

REINFORCE uses sampled complete returns and can have high gradient variance. The variance depends on the reward process, horizon, policy and baseline; horizon and discount alone do not specify its numerical scale. This week studies three related design choices:

1. Replace the Monte-Carlo return $G_t$ with a *bootstrapped* multi-step target $\hat{A}_t^{(n)}$ that changes the dependence on observed rewards and critic error (A2C, $n$-step returns).
2. Mix $n$-step estimators across all horizons into an estimator whose bias and variance depend on the critic and weighting (GAE).
3. Use local policy-distribution geometry and trust-region constraints to control updates, then distinguish the ideal improvement bound from the practical algorithm (Natural PG, TRPO).

*Intuition.* TRPO asks how to improve a policy while limiting changes in its action distribution. A noisy gradient can produce a damaging update; this is a risk, not an inevitable outcome of REINFORCE. The theoretical bound (Theorem 2.4.1) uses the maximum statewise divergence. Practical TRPO constrains an estimated average KL and uses approximate optimisation, so it does not inherit an unconditional monotonic-improvement guarantee.

## Why bother with TRPO if PPO exists?

TRPO motivates controlling policy change; PPO uses a clipped surrogate with a different optimisation procedure. The clip range changes incentives but does not impose TRPO's KL constraint. Studying the trust-region derivation helps separate those mechanisms before comparing their empirical behaviour.

**Common pitfall**

A common student error is to compute GAE *within* a mini-batch of one-step transitions *after* they have been shuffled. GAE is a recursive estimator along trajectory order; you must compute it on whole rollouts and then shuffle (treating episode boundaries with care). This is one possible failure among several; a disappointing training curve alone does not identify it.

# 2 Mathematical formulation

## 2.1 $n$-step advantages

Given a value function $V$, define the $n$-step return

$$\hat{R}_t^{(n)} \;=\; \sum_{k=0}^{n-1} \gamma^k r_{t+k} + \gamma^n V(s_{t+n}),$$

and the $n$-step advantage estimator

$$\hat{A}_t^{(n)} \;=\; \hat{R}_t^{(n)} - V(s_t) \;=\; -V(s_t) + \sum_{k=0}^{n-1} \gamma^k r_{t+k} + \gamma^n V(s_{t+n}).$$

At $n = 1$ this is a one-step TD advantage. A return extending to true termination becomes Monte Carlo; an unfinished rollout retains an endpoint bootstrap. Bias and variance depend on critic error and the data-generating process, so their ordering needs stated conditions.

**Definition 2.1.1** (Bellman residual / TD error)**.** $\delta_t \triangleq r_t + \gamma V(s_{t+1}) - V(s_t)$.

A short identity that we will use repeatedly:

$$\hat{A}_t^{(n)} \;=\; \sum_{k=0}^{n-1} \gamma^k \delta_{t+k}. \tag{†}$$

*Proof.* Expand: $\sum_{k=0}^{n-1} \gamma^k \delta_{t+k} = \sum_{k=0}^{n-1} \gamma^k r_{t+k} + \sum_{k=0}^{n-1} \gamma^{k+1} V(s_{t+k+1}) - \sum_{k=0}^{n-1} \gamma^k V(s_{t+k})$. The two value-function sums telescope into $\gamma^n V(s_{t+n}) - V(s_t)$. Done. □

## 2.2 Generalised Advantage Estimation (Schulman 2015)

**Theorem 2.2.1** (GAE closed form)**.** *For $\lambda \in [0, 1)$, with absolutely convergent sums, the exponentially-weighted average of $n$-step advantages*

$$\hat{A}_t^{\mathrm{GAE}(\gamma,\lambda)} \;\triangleq\; (1-\lambda) \sum_{n=1}^{\infty} \lambda^{n-1}\, \hat{A}_t^{(n)}$$

*equals*

$$\hat{A}_t^{\mathrm{GAE}(\gamma,\lambda)} \;=\; \sum_{l=0}^{\infty} (\gamma\lambda)^l\, \delta_{t+l}.$$

*Step-by-step derivation.* Substitute (†) into the definition:

$$\hat{A}_t^{\mathrm{GAE}} = (1-\lambda) \sum_{n=1}^{\infty} \lambda^{n-1} \sum_{k=0}^{n-1} \gamma^k \delta_{t+k}.$$

Swap the order of summation. The pair $(n, k)$ ranges over $1 \le n < \infty$, $0 \le k \le n-1$. Equivalently $0 \le k < \infty$ and $n \ge k+1$:

$$= (1-\lambda) \sum_{k=0}^{\infty} \gamma^k \delta_{t+k} \sum_{n=k+1}^{\infty} \lambda^{n-1} = (1-\lambda) \sum_{k=0}^{\infty} \gamma^k \delta_{t+k} \cdot \frac{\lambda^k}{1-\lambda} = \sum_{k=0}^{\infty} (\gamma\lambda)^k \delta_{t+k}.$$

The geometric series $\sum_{n=k+1}^{\infty} \lambda^{n-1} = \lambda^k/(1-\lambda)$ used $|\lambda| < 1$; the $\lambda = 1$ endpoint is defined through the residual sum or a limit, not by direct substitution into the weighted-horizon expression. It equals the Monte Carlo advantage at a true terminal endpoint; an incomplete rollout retains its final bootstrap value. □

*Remark* (Edges of the spectrum). $\lambda = 0$ gives $\hat{A} = \delta_t$ (one-step TD advantage, potentially biased by value-net error). $\lambda = 1$ recovers $\hat{A} = \sum_l \gamma^l r_{t+l} - V(s_t)$ for a completed episode; a segment ending at $T$ adds $\gamma^{T-t} V(s_T)$. The project uses $\lambda = 0.95$ as a starting hyperparameter; compare alternatives empirically for the chosen task.

See Schulman et al. (2015) for the original GAE experiments.

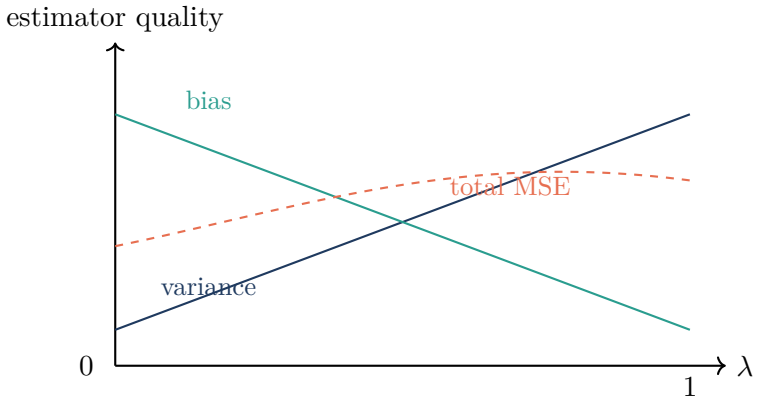



Figure 2.1: Schematic bias–variance intuition as $\lambda$ changes, not measured curves. The dashed line is an illustrative error curve, not the sum of the other two drawn curves. The best $\lambda$ depends on value error, reward noise and the task; 0.95 is a starting setting, not a universal optimum.

## 2.3 Natural Policy Gradient

The vanilla PG update $\theta \leftarrow \theta + \eta g$ lives in Euclidean parameter space, but a small step in $\theta$ can produce a huge step in policy space and vice versa. The right metric is the Fisher–Rao information:

**Definition 2.3.1** (Fisher information matrix)**.** For a parametric distribution $\pi_\theta(a \mid s)$ and a state distribution $d^\pi(s)$,

$$F(\theta) \triangleq \mathbb{E}_{s\sim d^\pi, a\sim\pi_\theta}\left[\nabla_\theta \log \pi_\theta(a \mid s)\, \nabla_\theta \log \pi_\theta(a \mid s)^\top\right].$$

Equivalently, $F = \mathbb{E}_s[\nabla^2_{\theta'} \mathrm{KL}(\pi_\theta \| \pi_{\theta'})|_{\theta'=\theta}]$ — the Hessian of the average KL at zero displacement.

The **natural policy gradient** (Kakade 2001) is

$$\theta \leftarrow \theta + \eta\, F(\theta)^{-1} g, \qquad g \triangleq \nabla_\theta J(\theta).$$

The natural-gradient tangent direction is invariant under smooth invertible reparameterisation (a finite Euler step need not be): if $\theta = \phi(\theta')$ smoothly, both $g$ and $F$ transform such that $F^{-1}g$ pulls back identically. Vanilla PG has no such guarantee.

## 2.4 TRPO (Schulman et al. 2015)

### Surrogate objective and policy improvement

Define the importance-sampled surrogate

$$L_{\theta_{\text{old}}}(\theta) \triangleq \mathbb{E}_{s,a\sim\pi_{\theta_{\text{old}}}}\left[\frac{\pi_\theta(a \mid s)}{\pi_{\theta_{\text{old}}}(a \mid s)} A^{\pi_{\theta_{\text{old}}}}(s,a)\right].$$

**Theorem 2.4.1** (Maximum-state divergence bound, Schulman 2015)**.** *Use the normalised discounted state distribution* $d^{\pi_{\text{old}}}(s) = (1-\gamma)\sum_{t\geq 0}\gamma^t \mathbb{P}(S_t = s \mid \pi_{\text{old}})$ *in the surrogate above, with exact old-policy advantages. Let* $\epsilon_A = \max_{s,a} |A^{\pi_{\text{old}}}(s,a)|$ *and* $\gamma < 1$*. A conservative form of the theoretical improvement bound is*

$$J(\theta) \geq J(\theta_{\text{old}}) + \frac{L_{\theta_{\text{old}}}(\theta)}{1-\gamma} - \frac{4\gamma\epsilon_A}{(1-\gamma)^2}\max_s \mathrm{KL}(\pi_{\text{old}}(\cdot \mid s)\|\pi_\theta(\cdot \mid s))\,.$$

*Sketch, full version in the paper.* The performance-difference identity expresses the return difference as $(1-\gamma)^{-1}\mathbb{E}_{s\sim d^{\pi_\theta}, a\sim\pi_\theta}[A^{\pi_{\text{old}}}(s,a)]$. The surrogate substitutes the old state distribution. A coupling argument bounds the resulting mismatch using the maximum statewise total variation, then a KL–total-variation inequality gives the displayed bound. This argument does not replace the maximum by an empirical mean KL. □

The practical TRPO method replaces the maximum-state constraint with an average KL under sampled old-policy states. This is a computational approximation and does not inherit the displayed monotonicity guarantee.

The bound suggests the optimisation problem

$$\boxed{\max_\theta\ L_{\theta_{\text{old}}}(\theta) \quad \text{s.t.} \quad \bar{\mathrm{KL}}(\theta_{\text{old}}\|\theta) \leq \delta.}$$

This is the **TRPO trust-region problem**.

## Local Lagrangian and natural-gradient step

Taylor-expand $L$ to first order and $\bar{\mathrm{KL}}$ to second order around $\theta_{\text{old}}$:

$$L_{\theta_{\text{old}}}(\theta) \approx g^\top(\theta - \theta_{\text{old}}), \quad \bar{\mathrm{KL}}(\theta_{\text{old}}\|\theta) \approx \tfrac{1}{2}(\theta-\theta_{\text{old}})^\top F(\theta-\theta_{\text{old}}).$$

With $u \triangleq \theta - \theta_{\text{old}}$ the problem becomes

$$\max_u\ g^\top u \quad \text{s.t.} \quad \tfrac{1}{2}u^\top F u \leq \delta.$$

Lagrangian: $\mathcal{L}(u,\lambda) = g^\top u - \lambda(\frac{1}{2}u^\top F u - \delta)$. KKT: $g - \lambda F u = 0 \Rightarrow u = (1/\lambda)F^{-1}g$. Plug back into the constraint:

$$\tfrac{1}{2\lambda^2} g^\top F^{-1} g = \delta \quad \Longrightarrow \quad \lambda = \sqrt{\frac{g^\top F^{-1} g}{2\delta}}.$$

Therefore the canonical TRPO step is

$$\boxed{\theta \leftarrow \theta_{\text{old}} + \sqrt{\frac{2\delta}{g^\top F^{-1} g}}\, F^{-1} g.}$$

## Conjugate gradient solve

In a deep net, $F$ is huge but $Fv$ is cheap to compute: take the second derivative of $\bar{\mathrm{KL}}$ at $\theta_{\text{old}}$ in the direction $v$. So instead of forming or inverting $F$, solve $Fx = g$ by conjugate gradient (CG), which takes only matrix–vector products. Ten CG iterations are typical.

**Conjugate-gradient solve of $Fx = g$**

**Init:** $x_0 = 0$, $r_0 = g$, $p_0 = r_0$.
**For** $k = 0, \ldots, K-1$:

1. $\alpha_k = (r_k^\top r_k)/(p_k^\top F p_k)$ (one Fisher–vector product).
2. $x_{k+1} = x_k + \alpha_k p_k$.
3. $r_{k+1} = r_k - \alpha_k F p_k$.
4. If $\|r_{k+1}\| < \varepsilon$ return $x_{k+1}$.
5. $\beta_k = (r_{k+1}^\top r_{k+1})/(r_k^\top r_k)$; $p_{k+1} = r_{k+1} + \beta_k p_k$.

**Output** $x \approx F^{-1} g$ in $K$ Fisher–vector products.

## Back-tracking line search

The quadratic approximation can lie; the actual surrogate at the proposed step might be *worse*. TRPO line-searches: shrink $\eta$ by $\rho^j$ for $j = 0, 1, 2, \ldots$ (typically $\rho = 0.5$) and pick the first $j$ such that (i) the empirical $\bar{\mathrm{KL}} \leq \delta$ and (ii) $L$ has increased. This enforces the measured surrogate and average-KL checks; even with line search, estimated advantages, finite samples, and the relaxed KL constraint do not guarantee monotone true return.

# 3 Algorithms

### A2C with GAE

**Synchronous A2C + GAE**

**Hyper-params:** $\gamma = 0.99$, $\lambda = 0.95$, $\eta = 3 \times 10^{-4}$, entropy coef $\beta = 0.01$, value coef $c_v = 0.5$, rollout length $T_R$.
**Repeat:**

1. Run $N$ parallel envs for $T_R$ steps. Collect $(s_t, a_t, r_t, \log \pi_t, V_\phi(s_t))$.
2. Compute $\hat{A}_t^{\text{GAE}}$ via the recurrence $\hat{A}_t = \delta_t + \gamma\lambda(1 - \text{done}_{t+1})\hat{A}_{t+1}$ — loop from $T_R - 1$ down to 0. Returns: $\hat{R}_t = \hat{A}_t + V_\phi(s_t)$.
3. Standardise $\hat{A}$.
4. Losses: $L_\pi = -\mathbb{E}[\log \pi_\theta \cdot \hat{A}]$, $L_V = \frac{1}{2}\mathbb{E}[(V_\phi - \hat{R})^2]$, $L_H = -\mathbb{E}[\mathcal{H}(\pi_\theta)]$.
5. Gradient step on $\theta, \phi$ jointly: $L = L_\pi + c_v L_V + \beta L_H$.

**Complexity per update:** $\mathcal{O}(NT_R(|\theta| + |\phi|))$.

### TRPO

**TRPO**

**Hyper-params:** target KL $\delta = 10^{-2}$, CG iters $K = 10$, line-search shrink $\rho = 0.5$, max 10 shrinks.
**Repeat:**

1. Collect a large rollout under $\pi_{\theta_{\text{old}}}$.
2. Compute GAE advantages $\hat{A}_t$ and the policy gradient $g = \nabla_\theta L$ at $\theta_{\text{old}}$.
3. Conjugate-gradient $x \approx F^{-1}g$ ($K$ Fisher–vector products of $\bar{\text{KL}}$).
4. Step size $\alpha = \sqrt{2\delta/(x^\top F x)}$. Proposed update $u = \alpha x$.
5. Back-track: $\theta \leftarrow \theta_{\text{old}} + \rho^j u$, smallest $j$ with $\bar{\text{KL}} \le \delta$ and $L$ increased.
6. Fit $V_\phi$ by regression on $\hat{R}$ (separate optimiser, often L-BFGS or Adam).

**Complexity:** dominated by $K$ Fisher–vector products; each is twice the cost of a policy forward+backward.

# 4 Implementation walkthrough

The trainer retains the sampled Gaussian latent action for its likelihood; only the action sent to the environment is clipped to the action-space bounds. Scoring a clipped action as the original draw would invalidate likelihood ratios. A true terminal removes the value bootstrap, while a time-limit truncation bootstraps from the final observation and stops the GAE trace before the reset episode. The common rollout helpers request same-step autoreset and recover the final observation supplied in its info dictionary.

## 4.1 GAE in ∼10 lines

```python
from common.rollouts import generalized_advantage_estimate

def compute_gae(rewards, values, dones, last_value,
                gamma=0.99, lam=0.95, *,
                next_values=None, terminals=None):
    # dones stops traces at termination OR truncation.
    # terminals only removes the bootstrap at true termination.
    # next_values uses final observations, never reset observations.
    return generalized_advantage_estimate(
        rewards, values, dones, last_value, gamma, lam,
        next_values=next_values, terminals=terminals,
    )
```

## 4.2 A2C update

```python
def a2c_update(batch, policy, value, opt, *, entropy_coef=0.01, value_coef=0.5):
    obs, acts, adv, ret = batch
    dist = policy(obs)                          # GaussianPolicy from common.networks
    logp = dist.log_prob(acts).sum(-1)
    entropy = dist.entropy().sum(-1).mean()
    v = value(obs).squeeze(-1)

    pi_loss = -(logp * adv).mean()
    v_loss = 0.5 * F.mse_loss(v, ret)
    loss = pi_loss + value_coef * v_loss - entropy_coef * entropy

    opt.zero_grad(); loss.backward()
    torch.nn.utils.clip_grad_norm_(
        list(policy.parameters()) + list(value.parameters()), 0.5)
    opt.step()
```

## 4.3 Fisher–vector product (TRPO core)

```python
def fisher_vector_product(kl_fn, params, v, damping=1e-2):
    # kl_fn() returns the scalar mean-KL between current policy and a detached
    ↪  snapshot
    kl = kl_fn()
    g_kl = torch.autograd.grad(kl, params, create_graph=True)
    g_kl_flat = torch.cat([gg.flatten() for gg in g_kl])
    gv = (g_kl_flat * v).sum()
    Hv = torch.autograd.grad(gv, params)
    Hv_flat = torch.cat([hh.contiguous().flatten() for hh in Hv])
    return Hv_flat + damping * v
```

# 5 Empirical results

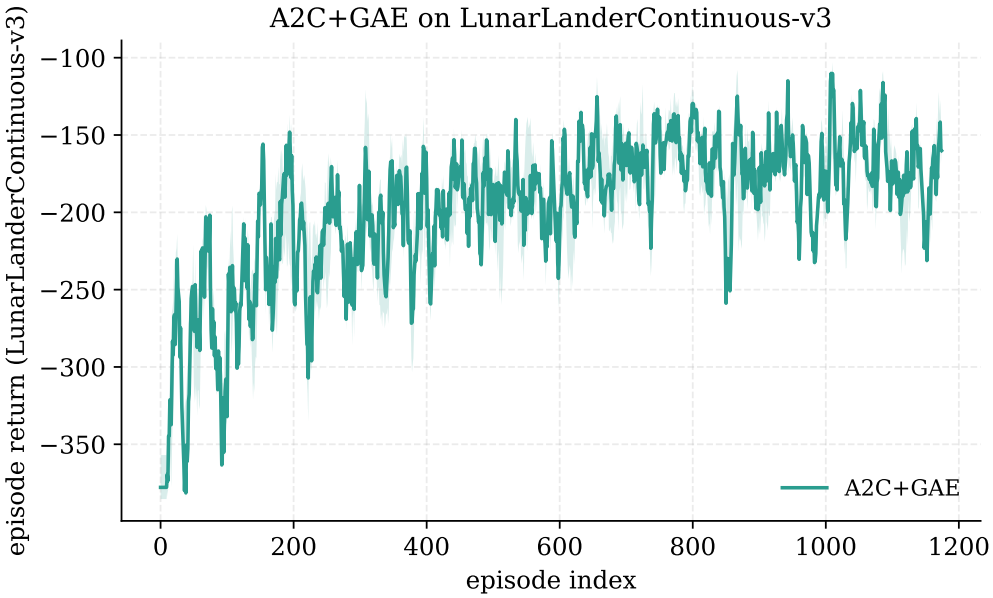



Figure 5.1: LunarLander-Continuous training curve. Synchronous A2C+GAE ($N = 8$ vector envs, $T_R = 128$) reaches positive returns within ∼200 updates. Mean and IQR over 3 seeds.

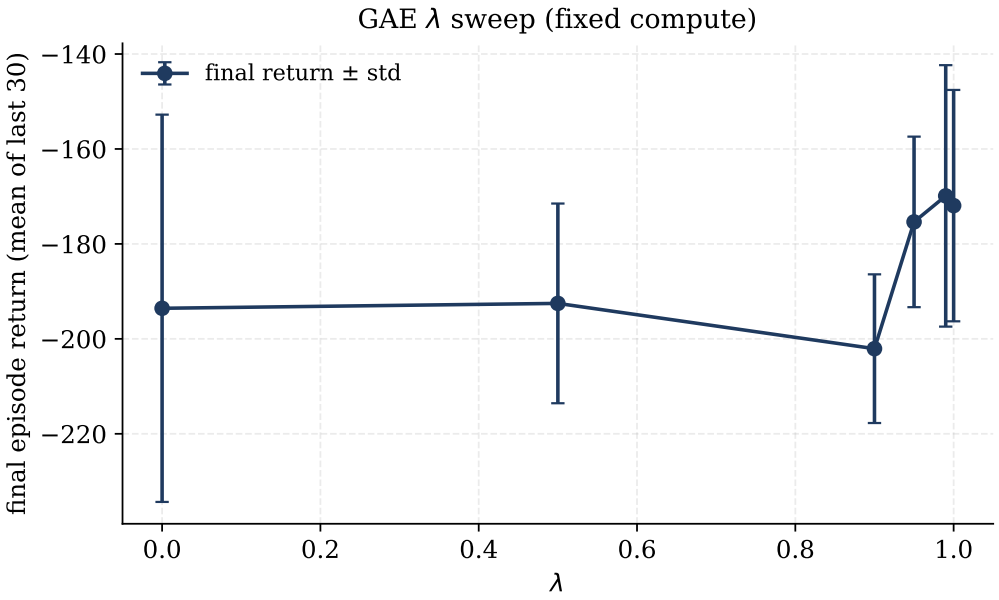



Figure 5.2: GAE bias–variance sweep. Final return after a fixed compute budget as a function of $\lambda \in \{0.0, 0.5, 0.9, 0.95, 0.99, 1.0\}$.

# 6 Safety angle

**Safety angle**

The KL trust-region in TRPO is not just an optimisation gadget — it is *the structural safety mechanism* of modern policy methods.

**What KL can bound.** For action distributions at one fixed state, Pinsker gives $\mathrm{TV}(p, q) \leq \sqrt{\mathrm{KL}(p\|q)/2}$, bounding every action-event probability difference at that state. If only $\mathbb{E}_{s\sim d}[\mathrm{KL}_s] \leq \delta$ is known, Jensen yields an *average* TV bound under the same $d$, not a per-person, rare-state, trajectory-risk or fairness guarantee. Empirical average KL adds sampling error. Policy proximity is an optimisation diagnostic, not safety itself.

**The pattern reappears everywhere downstream.**

- RLHF/PPO for LLMs: penalty $-\beta\mathrm{KL}(\pi_\theta\|\pi_{\mathrm{ref}})$ keeps the policy near the SFT model and discourages policy drift, but does not eliminate reward-model exploitation (Stiennon et al. 2020, Ouyang et al. 2022).
- DPO and friends (week 19): an optimal-policy reward reparameterisation takes the form $r(x, y) = \beta \log(\pi/\pi_{\mathrm{ref}}) + \beta \log Z(x)$. It follows from KL-regularised reward optimisation under its assumptions; it is not a hard KL constraint.
- Constrained RL (week 22): CPO is TRPO with an extra cost-constraint solved jointly via a closed-form 2D quadratic.

**Failure modes.**

1. *Damping mis-set.* A too-small CG damping causes the Fisher matrix to be effectively singular along low-variance directions and the line search collapses. Mitigation: set damping to $10^{-2} \cdot \mathrm{trace}(F)/|\theta|$ as a sane prior.
2. *KL divergence asymmetry.* TRPO penalises $\mathrm{KL}(\pi_{\mathrm{old}}\|\pi_{\mathrm{new}})$, not the reverse. This is mode-covering, not mode-seeking; in a multi-modal optimal policy you may end up in the wrong mode but with a reassuring small KL.
3. *Stale advantages on long-running envs.* If $V_\phi$ has not converged, $\hat{A}$ is biased and the trust region protects a wrong direction. Fit and diagnose the critic separately; a finite optimiser run does not guarantee an accurate value function or remove advantage bias.

# 7 Matching application

**Matching application**

**Trust regions for production policy updates in synthetic insurance.**
Imagine the current synthetic insurance lead-router has been A/B-tested and shipped — a known-quality policy $\pi_{\text{old}}$. We want to fine-tune it with the latest week of logged conversions, while measuring changes in its action distribution. A sampled average-KL constraint alone does not preserve an SLA or a fairness review.

**Direct translation.**

- **Surrogate $L$:** importance-weighted conversion reward, $\mathbb{E}_{s,a\sim\pi_{\text{old}}}[(\pi_\theta/\pi_{\text{old}})\cdot A]$, where $A$ is fitted from the logs.
- **Trust region:** an average-KL target of $10^{-3}$ is an illustrative setting. Under the same fixed context distribution, exact average KL bounds average action-event changes by $\sqrt{10^{-3}/2}\approx 0.0224$. It does not bound each candidate's probability change or preserve every exposure review.
- **Solver:** A2C+GAE plus a back-tracking line-search against the empirical KL is sufficient; we do not need the full conjugate-gradient TRPO machinery for the modest-sized policies in production.

**synthetic ATS connection.** For candidate-to-role matching, separately audit individual and group outcomes, context shift and uncertainty. Small measured mean KL can coexist with a large change for a rare candidate group. No compliance requirement or per-person guarantee follows from this surrogate alone.

# 8 Exercises

**E1** ($\star$) Prove identity (†).

**E2** ($\star$) Implement `compute_gae` and unit-test against the MC limit.

**E3** ($\star\star$) Show GAE at $\lambda = 0$ equals one-step TD; at $\lambda = 1$ equals MC.

**E4** ($\star\star$) Train A2C+GAE on LunarLander-Continuous.

**E5** ($\star\star$) Sweep $\lambda$; reproduce Figure 5.2.

**E6** ($\star\star$) Derive the Fisher matrix for a diagonal Gaussian policy.

**E7** ($\star\star\star$) Code a conjugate-gradient solver and verify $F^{-1}g$ on a toy MVN.

**E8** ($\star\star\star$, meaningful objective) Trust-region weekly retrain on logged matching data; distinguish fixed-context average slot-share bounds from per-context claims.

# 9 Further reading

See `extended_reading.pdf`. Five papers: Mnih et al. 2016 (A3C), Schulman et al. 2015 (GAE), Kakade 2001 (NPG), Schulman et al. 2015 (TRPO), Engstrom et al. 2020 (Implementation Matters). Sourced from `research/deeprl_core_sota.md` §2.2–2.5.